BUSINESS BANGERZ

Request for Proposal & Prototype Challenge

AI VOICE AGENTS

FOR THE BANGER PIPELINE

Business Bangerz writes original songs that make business messages stick — training, sales and marketing, policy rollouts, onboarding and culture. We are inviting engineers to spend 90 minutes building a prototype that puts an AI voice agent somewhere in that pipeline, and that either grows revenue or cuts cost. Ninety minutes with modern AI tooling is roughly a hackathon weekend of hand-written output — and it is a deliberate constraint. We are far more interested in what you choose to build than in how much of it you finish.

Issued by	Business Bangerz — businessbangerz.com
Document	Request for Proposal — AI Voice Agent Prototype Challenge
Format	A 90-minute live build sprint. AI-assisted development is expected.
Hard stop	90 minutes after kickoff. No extensions.
What you submit	A GitHub repository, submitted before the clock stops.
What happens next	A timed two-minute demo to the judges, in an assigned order.
How it is scored	Automated review of your repository against Section 6, plus human scoring of your demo.

CONTENTS

What is in this document

§	Section	What it covers
1	The challenge at a glance	The whole thing on one page, plus how to spend the 90 minutes.
2	About Business Bangerz	What the company sells, how it runs, and the constraint that shapes everything.
3	Why we are running this	The two acceptable outcomes, and what to think through before you build.
4	Where the current process hurts	Six named friction points, F1–F6.
5	The idea space	Three tracks of ideas, and how to choose between them.
6	Requirements	Mandatory and optional requirements, non-goals, and guardrails.
7	What you submit and how you present it	The repository, and the two-minute demo that follows.
8	Technical guidance	Stack, tooling categories, and repository hygiene.
9	Submission process	The form fields, private-repo access, and the rules of the road.
10	How submissions are scored	Both reviews, the rubric, bonuses, and deductions.
11	Terms of participation	IP, confidentiality, AI use, and general terms.
12	Questions	How and when to ask.
A	README structure	The sections we expect, in order. This one matters more than you think.
B	Submission checklist	Run this before you submit, and again before you present.
C	Glossary	Banger, Jam Sesh, Jukebox, and the rest.

START HERE

1 The challenge at a glance

If you read nothing else, read this page.

Question	Answer
What are we asking for?	Working code that shows how an AI voice agent could be used inside the Business Bangerz workflow. Pick one idea and build one slice of it properly.
How long do I have?	90 minutes from kickoff. You are expected to use AI coding tools, which makes that window roughly equivalent to a hackathon weekend of hand-written output — but it is still 90 minutes, and scoping is part of the test.
What counts as a good idea?	Anything you can credibly argue does one of two things: increases Business Bangerz revenue, or reduces the cost of producing a banger. If you can argue it, we are open to it.
Does the voice agent have to work?	No. A functioning voice agent is a strong plus, but a well-reasoned, partially-stubbed prototype beats a shallow one that happens to talk. Stubs are fine as long as you label them honestly — in your README and out loud in your demo.
Is there anything I must build?	Audio has to appear somewhere — captured, generated, or transformed. Which side it lands on is up to you. Everything else in Section 6.1 is satisfied by any reasonable prototype.
Who can respond?	Individuals or small teams. Every participant works within the same 90-minute window.
What do I submit?	A GitHub repository, public or private, through the submission form — before the clock stops. No slide deck, no recorded video, no written proposal.
And then what?	You demo it live. Two minutes, hard-timed, in an order assigned before demos begin. Be ready when your slot comes up.
How is it judged?	Two ways. An automated review reads your repository and matches it against the requirements in Section 6. The judges score the rubric in Section 10.3 from your demo — they are watching you present, not reading your source line by line.
How finished does it need to be?	Not very. One narrow flow that genuinely works beats a broad skeleton that does not. Unfinished is fine; unfinished and dishonestly labeled is not.

One rule above all others

Do not lead with the AI. Lead with the business outcome. We should be able to watch your two minutes, or read your README, ignore every technical detail in either, and still understand why Business Bangerz would want this.

1.1 Working inside 90 minutes

This is a sprint, not a take-home. The rubric in Section 10 is calibrated to the clock: nobody is expected to ship something complete, and the scoring explicitly rewards choosing a small enough problem over attempting a large one.

What the time box is actually testing

Scope discipline is a scored criterion, not an excuse. Anyone can start building all of Culture Mic in 90 minutes. Almost nobody can finish a slice of it that works and is worth looking at. The submissions that score well will be the ones where somebody decided, in the first ten minutes, what they were not going to build.

Use AI aggressively. Scaffolding, boilerplate, README drafting, test data generation — hand all of it to a model. Spend your own attention on the two things a model cannot do for you: deciding what to build, and telling the truth about what you built.

A suggested time budget

Ignore this entirely if you like. It exists because the most common way to lose this challenge is to spend 85 minutes coding and 5 minutes explaining.

Minutes	Phase	What you should walk out of it with
0–10	Choose	One concept, one friction point (F1–F6), one outcome (revenue or cost), and an explicit list of what you are NOT building.
10–20	Design the slice	The single path you will build, end to end, on paper. Decide now which parts will be real and which will be fixtures.
20–65	Build	That one path, working. Resist every temptation to add a second one.
65–75	Pre-bake	Produce and save the finished result you will cut to in your demo. Do this while you still have time, not while the judges are watching.
75–85	README, then submit	Appendix A, then the form. Verify the repo actually clones.
85–90	Rehearse	Say your two minutes out loud once. It is the only way to find out it is really four.

The one pre-work rule

Have your environment ready — editor, AI tooling, API keys, whatever starter template you would reach for on any project. That is expected. But do not write project-specific code before kickoff. Your repository should begin at kickoff, and commit timestamps are checked.

2 About Business Bangerz

Business Bangerz (**businessbangerz.com**) writes original songs for businesses. Not stock music, not background beds — actual songs, with lyrics, written around a specific message a company needs its people to understand, remember, and act on. The tagline on the site says it plainly: make it a banger, not a deck.

2.1 What the company sells

Use case	What it looks like
Training	Strategy offsites, HR and compliance training, new tool rollouts — turned into a song people remember instead of a deck they forget by lunch. Released examples include a harassment-prevention training pop song.
Sales & marketing	Launch hype, product drops, all-hands openers — a track that gets a room fired up.
Policy rollouts	Expense process changes, PTO policy updates, and other dry-by-default announcements.
Onboarding & culture	Day-one theme songs for new hires, team milestone tributes, culture pieces.

2.2 How the business currently runs

- **Front door:** a free 20-minute "Jam Sesh" — a booked consultation where the client explains what is coming up and the team figures out what their banger sounds like.
- **Pricing:** scoped per project. Published rates are explicitly starting points, not quotes.
- **Audience building:** "Banger of the Week," a free weekly newsletter that also unlocks the Jukebox.
- **The Jukebox:** a members-only library of every released banger, tagged by use case and genre — Culture / Indie Folk, AI Adoption / Indie Pop, Harassment Prevention Training / Pop, Strategy & Planning / Pop, and so on. Sign-up gates playback.
- **Rights:** by default Business Bangerz retains the right to reuse songs, lyrics, recordings, and videos it produces — including client work — for promotional and portfolio purposes, unless a written agreement says otherwise.
- **Who runs it:** founder Matthew Zimmer — mechanical engineering degrees from Michigan, an MBA from UC Irvine, seven years scaling manufacturing at Niagara Bottling, then founder of StackTek and cheddr. He now also teaches business and AI at Miramar College and the University of San Diego.

> **The single most important operating constraint**
>
> **Business Bangerz is a very small operation.** Any solution that only works if the company hires a five-person ops team is not a solution. Assume the person running your tool is also writing the song, taking the client call, and shipping the newsletter that week. Build accordingly.

2.3 The technical surface we can observe

The following is inferred from the public site, not confirmed. Treat it as a hint about where your work would eventually plug in, not as a requirement.

- The marketing site and Jukebox appear to be a Next.js / React application.

- Song audio, video, and thumbnail assets are served from Supabase storage.
- Membership appears to use passwordless email sign-in, consistent with Supabase auth.
- Jam Sesh booking uses an embedded third-party scheduler.
- Songs already carry structured metadata — title, use-case tag, genre tag, thumbnail — which suggests a song record already exists in a database somewhere.

3 Why we are running this

Business Bangerz already uses AI in production. That is not the question. The question is where a conversational voice agent belongs in the workflow — and honestly, we are not sure. We have opinions and a consultant report full of ideas, but we would rather see prototypes than argue about slides.

So we are running this as an open challenge with a deliberately wide aperture, a deliberately narrow success test, and a deliberately short clock. Ninety minutes is not enough time to build the right answer. It is enough time to show us that you found it.

3.1 The two acceptable outcomes

Every submission should move one of these two needles. Not both. Pick one, name it in your README, and say it in your demo.

Outcome A — Increase revenue	Outcome B — Reduce cost
What a credible revenue argument looks like: • It creates a new billable line item or product tier. • It converts more free Jam Seshes into paid projects. • It justifies a higher price per project. • It turns one-off song orders into recurring revenue. • It lets the company serve more clients without adding people. • It creates an upsell after delivery, not just before it.	**What a credible cost argument looks like:** • It removes hours from producing a single banger. • It cuts the number of revision rounds per project. • It reduces scheduling and coordination labor. • It prevents rework caused by a bad or incomplete brief. • It replaces a manual search-and-audition step with a fast one. • It collects material from many employees without many meetings.

3.2 What to think through before you start building

You do not have to write any of this down or submit it. Nobody is grading a business case document. But the difference between a submission that lands and one that does not is usually whether somebody spent ten minutes on these questions before opening an editor.

Ten minutes of thinking, none of it a deliverable

Which outcome? Revenue or cost. If you cannot pick, you do not yet have an idea.

What goes wrong today, and what does it cost when it does? Two sentences in your head.

Does the arithmetic actually work? If a project takes roughly N hours and this removes M of them, across a handful of projects a month — is that a number worth anyone's attention? Rough is fine. Zero is not.

What does one run of this cost to operate? A cost-saving tool with expensive API calls behind it may not save anything.

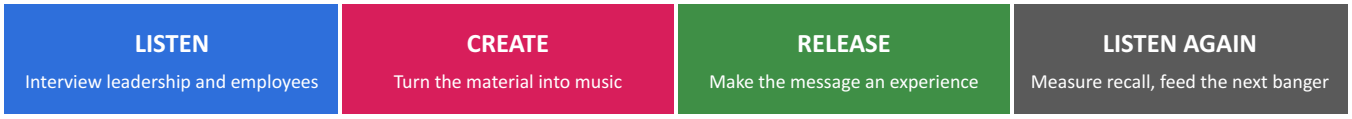
What would have to be true for this to work? The two assumptions that, if wrong, sink it.

Does it survive the small-team constraint? See Section 2.2.

What are you deliberately not building? This one you do write down — in your README.

3.3 The loop we are trying to close

The strategy report accompanying this RFP frames the target operating model as a loop. It is a useful map for locating your idea.



Today the company is strong at CREATE and RELEASE. LISTEN is a manual conversation. LISTEN AGAIN barely exists. That asymmetry is where most of the opportunity lives — but you are not obligated to agree with us.

4 Where the current process hurts

These are the friction points we have identified. You do not have to attack one of these — but if you attack something else, explain in your README what you saw that we did not. Reference these by ID.

ID	Friction	What actually goes wrong
F1	Intake quality	Clients describe what they want in deck language. The real objective — what people currently misunderstand and what leadership wants them to do differently — stays buried. Stories, must-say phrases, banned wording, and pronunciations of internal jargon get missed until it is expensive to fix.
F2	Revision friction	Feedback arrives as prose in email: "the chorus works, verse two feels corny, I liked the drums in the other version." It is untimestamped, unprioritized, and ambiguous. Someone has to translate it into production notes by hand.
F3	Sound search	Finding the right instrument, texture, or sample means describing music in words, then auditioning options one at a time. Most clients cannot describe a sound. Many can hum it.
F4	Participation logistics	The most ownable songs include the client's own people — their voices, their phrases, the sounds of their workplace. Collecting that from dozens of employees is currently a scheduling problem that scales badly.
F5	No proof of impact	After delivery there is no evidence the message landed. That makes repeat purchase a matter of taste rather than results, and makes premium pricing hard to defend.
F6	No reusable memory	Every project starts from zero. Client language, brand voice, pronunciations, approved phrasing, and past feedback are not captured in any structured, reusable form.

5 The idea space

Three tracks. You may draw from any of them, combine them, or ignore all of them and bring your own. What you may not do is build five shallow things instead of one convincing one.

5.1 Track A — Ideas Business Bangerz has already brainstormed

These are the company's own starting hunches. They are not ranked, and none of them is a requirement.

#	Idea	The version of this we would find interesting
A1	Better lyric and message input	A conversational agent that interviews the client instead of handing them a form — probing vague answers, pulling out the behavior change behind the request, and emitting a structured brief the writer can actually use. Attacks F1 and F6.
A2	Capturing and cloning team member voices	Consented capture of employee voices for cameos, tags, choruses, or personalized variations. The interesting design question is where cloning stops being delightful and starts being uncanny — and how consent is handled. Attacks F4.
A3	Office sounds as musical elements	Prompting employees to record the sounds of their workplace — machines, doors, keyboards, forklifts, packaging lines — and turning those into percussion or texture. A sonic signature that could only belong to that company. Attacks F4.
A4	Hum-to-search across a sound library	Sing, hum, or beatbox a shape and get back matching instruments, samples, or presets, instead of describing timbre in words. Attacks F3.
A5	Live vocal overlay onto a track	Capturing live vocals over an existing bed — at an event, an all-hands, or a booth — and getting usable material out the other end without a studio. Attacks F4 and possibly creates a new billable experience.

5.2 Track B — Concepts from the attached strategy report

An outside consultancy delivered the "AI Voice Agent Opportunity Report" that accompanies this RFP. It contains 21 numbered concepts and a prioritization matrix. Its three top recommendations are:

Concept	Role	Report priority
Banger Brief	A conversational creative director that produces a structured brief from a natural interview.	Build first
Culture Mic	Interviews employees to surface authentic language, stories, inside jokes, and culture signals.	Pilot
Banger Recall Test	Measures comprehension and recall after release, so effectiveness can be proven.	Moat

The report also proposes Revision Room, Message Miner, Spoken Lyric Workshop, Pronunciation Capture, Department Remixes, Employee Cameos, The Company Choir, Sonic DNA Capture, Live Banger Booth, The Culture DJ, Banger Companion, Town Hall to Track, The Company Mixtape, Recognition Radio, Global Banger, and M&A Culture Mashup.

> **A warning about Track B**
>
> The report is a strategy document, not a spec. Rebuilding its top recommendation exactly as described is a safe submission, not an impressive one. If you take a report concept, we expect you to sharpen it, disagree with part of it, or find the hard problem inside it that the report glossed over. Reviewers have read it too.

5.3 Track C — Your own idea

Bring something we have not thought of. This track carries the most upside on the creativity criterion and the most risk on everything else, because you have to establish the problem before you can claim to have solved it. If you go this route, spend the first paragraph of your README explaining what you saw in the workflow that made you build this.

5.4 How to choose

- **Pick one primary concept.** Name it. Everything else in your repo is supporting material.
- **State the friction ID you are attacking** (F1–F6, or describe your own).
- **State the outcome** — revenue or cost.
- **Prefer the boring, high-frequency problem** over the flashy, once-a-year one. Something that touches every project beats something that touches one project a year — unless the once-a-year one is enormous, in which case say so.

6 Requirements

These are the requirements your code is matched against. Mandatory requirements use "shall" and apply to every submission regardless of which concept you chose. Optional requirements use "should" or "may" and are where the differentiation happens.

How to read this section

Every mandatory requirement is deliberately concept-agnostic. Whatever you build — a brief interviewer, a hum-to-search tool, a recall test — it touches audio somewhere, takes an input, turns it into something structured, runs from somewhere, and is described in a README. Those are the mandatory ones, and all six are achievable inside 90 minutes.

MR-1 is the only requirement that constrains what you build, and it is deliberately loose. Audio has to appear somewhere in your workflow — but it can be on either side. A system that listens and never speaks satisfies it. So does one that takes typed input and speaks the result. So does one that never talks to a person at all and instead searches, slices, or generates sound. What we will not accept is a text-in, text-out tool with no audio anywhere in it.

Skipping an optional requirement costs you nothing. In the automated pass, an optional requirement only ever adds to your score when it matches — an unmatched one subtracts nothing at all. Most of the thirteen will not apply to your concept, and ignoring those is the correct move, not a sacrifice.

That is not an invitation to farm them. The automated pass cannot tell a working capability from a well-named empty one — but the judges see its report next to your demo, and a repository full of capabilities you never mention in two minutes reads as padding rather than range. It will cost you more on scope discipline than it gains you anywhere else.

6.1 Mandatory requirements

ID	Requirement
MR-1	The system shall capture, generate, or transform audio as part of its workflow.
MR-2	The system shall accept an input supplied by a person — spoken, recorded, or typed — as the entry point to the workflow.
MR-3	The system shall transform that input into a structured, machine-readable artifact that a downstream production step could consume.
MR-4	The system shall expose a single runnable entry point that executes the primary workflow from input through to output.
MR-5	The README shall state the concept name, the friction point it addresses, and the business outcome it targets.
MR-6	The README shall state which parts of the system are functional and which are stubbed, mocked, or simulated.

6.2 Optional requirements

Pick the ones your concept actually needs.

ID	Requirement
OR-1	The system should transcribe spoken audio into text.
OR-2	The system should generate a spoken response using synthesized speech.
OR-3	The system should conduct a multi-turn conversation, asking a follow-up question when an answer is vague or incomplete.
OR-4	The system should handle interruption, silence, or background noise during a spoken exchange.
OR-5	The system should capture and store the pronunciation of company-specific names, acronyms, and internal jargon.
OR-6	The system should map its output onto the fields of a song brief: objective, audience, required messages, stories, tone, references, and constraints.
OR-7	The system should record explicit consent before capturing or reusing any person's voice, and should record how long that audio is retained.
OR-8	The system should search a library of sounds or samples using an audio query rather than a text description.
OR-9	The system should collect contributions from multiple participants and aggregate them into a single result.
OR-10	The system should estimate and report the cost of a single session.
OR-11	The system should log the events needed to measure whether the targeted business outcome is being achieved.
OR-12	The system should read credentials and configuration from the environment rather than from committed source, and provide an example environment file listing every variable it needs.
OR-13	The system may run in an offline demonstration mode using recorded fixtures, without requiring paid API credentials.

6.3 Explicitly out of scope

Do not spend your time on these. They will not earn points.

- Production hardening — auth hardening, billing, multi-tenancy, compliance certification, uptime engineering.
- The quality of generated music itself. We are not judging your song. Music generation may appear in your prototype, but craft belongs to Business Bangerz.
- Rebuilding or redesigning the marketing website.
- Anything that requires Business Bangerz to hire an operations team to use it.
- Polished visual design for its own sake. Clear beats pretty; pretty and unclear scores worse than plain and clear.
- Tests, CI, containerization, and error handling beyond the happy path. Not expected in 90 minutes and not scored.
- Slide decks. A sentence or two of setup in your demo is expected; a deck is not.
- **Breadth.** Building a little bit of three ideas is the single most reliable way to score badly on this challenge.

6.4 Guardrails — voice, consent, and content

Voice is biometric data in several jurisdictions, and this challenge specifically invites ideas about capturing employee voices. We take that seriously and so should your design.

- **Do not clone a real person's voice without their documented consent.** Use your own voice, a synthetic voice, or a properly licensed voice dataset.
- **Never use the voice of an executive, employee, celebrity, or public figure** as a demo subject.
- **If your concept involves employee voice capture, build the consent flow.** Who is asked, what they are told, what they can decline, how they withdraw, how long the recording is kept. A design that shows awareness of BIPA, CUBI, and GDPR biometric provisions earns bonus points; you are not expected to produce legal analysis.
- **No copyrighted commercial recordings** in your prototype. Use original, public-domain, or properly licensed audio, and say which in your README.
- **Check the commercial-use terms** of any generative model or API you build on, and note them in your README.
- **Use synthetic or clearly fictional client data.** Do not use a real company's internal policy documents, and do not use real Business Bangerz client material.

7 What you submit and how you present it

7.1 The repository

One GitHub repository, public or private, submitted through the form before the clock stops. There is no deck, no recorded video, no written proposal, and no business case document.

Everything lives in the repository: your source code, and a README following the structure in Appendix A. If your code needs credentials, include an example environment file listing the variables it expects — never the real values. Nothing else needs to be in there.

> **Why the README matters more than usual here**
>
> The automated review reads and matches your documentation the same way it reads and matches your source code. A README section that plainly states what your system does is a real, standalone match candidate — sometimes a stronger one than the code that does the work. A repository with excellent code and a thin README will underperform one where both are clear.
>
> That is not a reason to write a novel. It is a reason to give the README a real slot in your ninety minutes rather than whatever is left at minute 89.

7.2 The two-minute demo

When the clock stops, everyone demos. Two minutes each, hard-timed, in an order assigned before demos begin. This is the part the judges actually score — they are watching your demo, not reading your source line by line. Your repository is what the automated review reads.

Length	Two minutes maximum, timed. At 2:00 you are done, mid-sentence or not.
Order	Set by the organizer and announced before demos begin. Be ready when your slot comes up — the clock does not wait for you to find a cable.
Setup	You present from your own laptop, plugged into the room TV. Bring whatever adapter your machine needs. Audio comes out of the TV, so check that your laptop is actually routing sound there and not to its internal speakers.
Live or pre-baked	Either. Live demos are appreciated and will earn your project extra attention, but they are not required and not worth the risk if the thing is fragile.
Questions	Informal and at the judges' discretion. You may get a follow-up, you may not. Do not build your two minutes around the assumption that you will be asked.
What to cover	What it is, who it is for, and what it does. Then show it.

Organize it like a baking show

Show us how it is done, then cut to the pre-baked result. Walk through enough of the process that we understand what is happening — the interview starting, the audio going in, the search running — then pull the finished tray out of the oven. Nobody wants to watch a model think for forty seconds, and you do not have forty seconds to spare.

Produce that pre-baked result during the build window, not during the demo. Save the output, the transcript, the generated brief, the matched sample — whatever the end of your flow produces — while you still have time to fix it if it comes out wrong.

Say what is real and what is staged. Cutting to a saved result is expected and costs you nothing. Presenting a saved result as if it just happened live is misrepresentation, and it is scored as such.

Test your audio before demos start

This is an audio challenge presented through a television. Plug in during the build window or in the gap before demos, play something, and confirm sound is coming out of the TV rather than your laptop. A demo the room cannot hear is a demo that did not happen, and your two minutes will not be extended while you hunt through sound settings.

Two minutes is roughly 300 spoken words. That is one paragraph of setup and one thing shown working. It is not a tour of your architecture, a walkthrough of your file tree, or a recap of the RFP back at the people who wrote it. Assume the judges know the brief and want to see what you did with it.

8 Technical guidance

None of this is mandatory. It exists so you spend your time on the interesting problem instead of on tooling decisions.

8.1 Stack

Use whatever you are fastest in. That said, if you want maximum credit on integration fit, the observed Business Bangerz surface is TypeScript / Next.js with Supabase for storage and auth. A prototype that could drop into that world has a shorter path to reality than one that could not.

8.2 Voice and language tooling

The following are examples of the categories of tool that exist, not endorsements or requirements. Choose freely; disclose what you used.

Category	Examples of what exists in this space
Realtime speech-to-speech	Vendor realtime voice APIs that handle listening, reasoning, and speaking in one connection.
Assembled pipelines	Speech-to-text, then a language model, then text-to-speech — stitched with an orchestration framework. More moving parts, more control over each step.
Voice agent platforms	Hosted platforms that manage turn-taking, interruption handling, latency, and telephony so you do not have to.
Telephony and room infrastructure	Phone-number and real-time-media layers, useful if your concept involves calling employees or running a session in a room.
Voice synthesis and cloning	Vendors offering both stock synthetic voices and consented voice cloning. See the guardrails in Section 6.4 before you use the latter.
Audio analysis for hum-to-search	Pitch and melody extraction, audio embeddings, and vector similarity search — plus openly licensed sample libraries with APIs.
Reasoning and structuring	Any general-purpose language model, used to turn a messy transcript into a structured brief, prioritized notes, or tagged output.

8.3 Repository hygiene

- Commit as you go rather than once at the end. In a timed event, commit history is how we verify the work happened inside the window.
- Keep generated artifacts, dependency directories, and large binaries out of the repo. They are excluded from the automated review anyway, and they slow down anyone who does open your code.
- Do not commit secrets. Provide an example environment file instead.
- If you fork or vendor someone else's work, say so plainly in the README. Building on open source is expected; presenting it as your own is not.
- Volume does not help. Code unrelated to anything in Section 6 is surfaced explicitly as unexplained code in the automated report. A small, focused repository where everything ties back to a requirement reads better than a large one where most of it does not.

9 Submission process

Submit through the form at **https://submit.vibecraft.works**. Emailed submissions are not reviewed. Submit before the clock stops — budget five minutes for it.

9.1 What the form asks for

Field	Required?	Notes
Your name	Required	If you worked as a team, list everyone.
Your email	Required	Where we send results.
GitHub repository URL	Required	The full URL, e.g. https://github.com/your-handle/your-repo. Public or private are both fine.
Branch, tag, or commit	Only if not main	Leave blank and we review main as it stands at the hard stop. Naming a tag or commit is the safer choice — it pins exactly what we look at.
Personal access token	Only for private repos	Required only if your repository is not public. See Section 9.2 for how to scope it.

9.2 If your repository is private

Create a GitHub fine-grained personal access token scoped to that single repository, with read-only permissions on Contents and Metadata, and a short expiry. Paste it into the form. We use it to clone and read your code, and we delete it once evaluation closes — revoke it yourself as soon as results are announced.

> **Token safety**
>
> Never give us a classic token with broad scopes, an organization-wide token, or a token that can write. If a submitted token has more access than described above, we will ask you to replace it before review. Do not put tokens anywhere in the repository itself.

9.3 Rules of the road

- One submission per person or team. If you submit twice, we review the later one submitted before the hard stop.
- The hard stop is enforced against the commit you point us at. If you named a tag or commit, its timestamp governs; if not, we take main as of the 90-minute mark. Commits after that mark are not reviewed.
- **Do not force-push or rewrite history after submitting** unless you have tagged the exact commit you want reviewed.
- Keep the repository accessible until results are announced. A repo we cannot clone is a repo we cannot score.
- You may keep working after you submit — but we review the hard-stop state, not the latest state.
- Demos begin once the clock stops. The organizer sets the running order and announces it beforehand; you present from your own laptop on the room TV, and you need to be ready at your slot.

10 How submissions are scored

10.1 Two reviews, one score

> **Disclosure: submissions are graded by a combination of AI and human review.**
>
> Every submission is first processed by an automated review that reads the repository and matches its contents against the requirements in Section 6. It produces a structured assessment, with the evidence for each match, before any person sees your work.
>
> The judges then score the rubric in Section 10.3 from your two-minute demo. They are watching you present, not reading your source line by line — though the repository is available to them, and they will look at it when a demo raises a question. The automated assessment is an input to their judgement, never a substitute for it. Where the two disagree materially, the human score governs.
>
> Practical implication: the machine reads your repository and the people watch your demo. Both matter, and they reward different things. Clear naming and an explicit README carry the automated pass; two well-spent minutes carry the human one.

10.2 How the automated review reads your repository

The automated pass uses VibeCheck. It does not run your code — it compares the text of your repository, including your documentation, against the requirement text in Section 6, and matches each requirement to its single best-fitting piece of your codebase. The full specification is published at **https://vibecraft.works/specs/vibecheck.md** — reading it is worth five of your ninety minutes.

The parts that change how you should write:

- **Your README is treated as code.** It is chunked and matched exactly like a source file, and it often produces the strongest match in a repository. A capability you built but never described anywhere is weaker than one you built and stated plainly.
- **Each requirement matches one place, not many.** A capability scattered across a dozen loosely-related snippets dilutes its own signal. Give each capability a single, well-named function, class, or module that clearly owns it.
- **Name things the way this document talks about the problem.** Matching is semantic rather than keyword search, but a name that echoes the vocabulary in Sections 4 and 6 gives a much stronger signal than one that only makes sense to someone already inside your codebase.
- **Write comments about intent, not mechanics.** "Retries with backoff so a flaky connection does not drop the interview" reads like a requirement. "Loop over the array" does not.
- **A missed mandatory requirement grades the whole run a Fail.** Your numeric score is left alone, but the grade is Fail regardless of how well everything else matched. That is why all six mandatory requirements in Section 6.1 are small and concept-agnostic — make sure each one has an obvious home in your repo before you polish anything else.
- **Optional requirements are upside only.** A match adds its weighted share to your score; an unmatched one takes nothing away. Reach for the ones your concept actually needs and leave the rest alone without worrying about the cost.
- **Keyword stuffing does not work.** Piling requirement language into comments above code that does not do that thing reads as noise. And unexplained code — volume unrelated to any requirement — is surfaced explicitly in the report.

What the automated pass does not do

It does not execute, test, or verify that your code works. A well-named, well-documented function that is broken can still match. That is exactly why a human reads everything afterward — and why writing clearly and writing correctly are two separate jobs. Do both.

10.3 The human rubric

100 points, plus up to 10 bonus points. Scored from your two-minute demo, with the repository available as reference. Weighting reflects what this RFP values — thoughtfulness, creativity, and execution — and is calibrated to a 90-minute build. Volume of code is not a criterion. Judgement is.

Criterion	Pts	What we are actually assessing
1. Business thinking and outcome alignment	20	Does the demo name one outcome — revenue or cost — and is the reasoning behind it clear? Does the concept attack a real friction point rather than an imagined one? Does it respect the small-team constraint in Section 2.2? We are not grading a business case; we are looking for evidence that someone thought before building.
2. Solution design and creativity	25	Is this a genuine insight about the workflow, or a generic voice-agent demo pointed at a music company? Did the submitter find the hard problem inside the idea? Is the design shaped by the constraints of this specific business? Originality and taste are rewarded; so is a familiar idea executed with an unfamiliar angle.
3. Execution and scope discipline	25	Does the primary path work end to end? Was the slice well chosen for the time available — small enough to finish, meaty enough to matter? Is the working scope honestly represented: is what is real actually real, and is what is staged or stubbed said out loud? A narrow flow that works outscores a broad one that does not.
4. Voice agent integration	10	Is a voice agent meaningfully in the loop? Does the conversational design handle vagueness, interruption, and mess, or only a scripted happy path? Is voice the right modality here, or bolted on? A thoughtfully stubbed agent with excellent conversation design can outscore a working one with none.
5. Demo and communication	20	Do the two minutes land? Is the value obvious to a non-engineer within the first twenty seconds? Was the demo shaped for the time it had, or did it run out of clock mid-thought? Is the submitter straightforward about what does not work? Polish is not the point — clarity is. (Code legibility is measured separately by the automated pass; see Section 10.2.)

10.4 Scoring bands

Each criterion is scored into one of four bands, then converted to points.

Band	% of criterion	What it means
Exemplary	90–100%	Would change how we think about this part of the business. We would want to talk to this person regardless of the outcome. Rare, and does not require a finished product.
Strong	70–89%	Clearly meets the intent, well-executed, a few gaps or unexamined assumptions.
Adequate	40–69%	Present but generic, thin, or unexamined. Meets the letter of the requirement. Most 90-minute submissions will land here on at least one criterion, and that is fine.
Insufficient	0–39%	Missing, broken, misrepresented, or off-target.

10.5 Bonus points (up to +10)

Bonus	Pts	Criteria
Responsible voice design	+4	A consent flow that is built, not mentioned. Clear handling of retention, withdrawal, and what an employee is told before their voice is captured.
Integration realism	+3	Demonstrably fits, or has a credible documented path into, the existing Next.js / Supabase surface — including a realistic estimate of adoption effort.
Measurement built in	+3	Instrumentation that would let Business Bangerz prove the targeted outcome six months from now, not just assert it today.

10.6 Deductions and disqualification

Issue	Consequence
Secrets or live credentials committed to the repository	–5 points and a notification to you
Stubbed, staged, or non-functional components presented as working	–10 points, and a material hit to the execution criterion
Not ready to present when your demo slot comes up	Your slot is forfeited and the rubric is scored from your repository instead — a worse deal for you than two rough minutes
Project-specific code committed before kickoff	–10 points, or disqualification if substantial
Undisclosed use of someone else's substantial work as your own	Disqualification
Cloning a real person's voice without documented consent	Disqualification
Use of real client data, or copyrighted commercial recordings	Disqualification
Submission after the 90-minute hard stop	Disqualification

10.7 Review sequence

- **Demos.** Two minutes each, in the order the organizer sets, immediately after the clock stops. The judges score the rubric from what they see, and may ask a follow-up question.
- **Eligibility screen.** We confirm the repository clones and that the mandatory requirements in Section 6.1 have a home in it.
- **Automated assessment.** Requirement matching against Section 6, with evidence cited from the repository.
- **Results.** Scores returned to submitters.

11 Terms of participation

> **This section is a starting draft, not legal advice.**
>
> Have counsel review it before this RFP is distributed, particularly if an award, payment, or downstream engagement is attached to it.

11.1 Intellectual property

- You retain ownership of everything you create and submit.
- By submitting, you grant Business Bangerz a non-exclusive, royalty-free right to access, clone, run, evaluate, and internally discuss your submission for the purposes of this RFP.
- Business Bangerz may reference your submission — including screenshots and quotes from your README — in internal materials and in feedback shared with other participants, unless you state otherwise in your README.
- **Any award, engagement, purchase, or transfer of rights is subject to a separate written agreement.** Submitting does not transfer ownership, and reviewing does not create an obligation to engage.
- If your submission incorporates third-party or open-source components, you are responsible for confirming their licenses permit this use, and for disclosing them in your README.

11.2 Confidentiality

- This RFP and the attached strategy report are confidential. Do not publish, redistribute, or post them.
- You may make your own submission repository public after results are announced, provided it does not reproduce the strategy report or any confidential Business Bangerz material.
- You may discuss your own participation publicly. Do not reproduce this RFP or the strategy report in doing so.

11.3 Use of AI tools

- Using AI coding assistants and models is expected and carries no penalty whatsoever. This is an RFP about AI; pretending you built it by hand would be strange.
- Disclose which tools and models you used, and roughly for what, in your README.
- You remain responsible for everything in your submission, including code you did not write line by line. "The model wrote it" is not a defense for a licensing violation or a committed secret.

11.4 General

- Business Bangerz is not obligated to make any award, select any submission, or proceed with any engagement.
- Participants bear their own costs, including any API spend.
- Business Bangerz may amend this RFP; material amendments will be sent to all participants.

12 Questions

Questions before the event go to the organizer, and answers are shared with every participant so that nobody gains an advantage by asking. During the 90-minute window, questions go to the event channel and answers are broadcast to everyone.

Good questions to ask us: anything about the current tooling, the real numbers behind a cost argument, what a typical project actually involves, or whether an idea you are considering has already been tried. We would rather answer than have you guess — and given the clock, ask before kickoff wherever you can.

APPENDIX A

A README structure

Your README is what the automated review reads, the same way it reads your source code — and the judges will open it whenever your demo raises a question. Use these sections, in this order. Drafting it with an AI tool is expected — but read what it wrote, because you own every claim in it, and a README that oversells what exists is the fastest way to lose points on honesty.

Section	What goes in it
1. What this is	Two sentences. The concept name, and what it does for Business Bangerz.
2. The outcome it targets	Revenue or cost, the friction ID from Section 4, and one or two sentences on the mechanism — why this saves money or makes money.
3. What actually works	An honest inventory. Which parts are real, which are stubbed, which are faked. Write it before you write your demo script — it is the same list. It satisfies MR-6.
4. How it works	A plain-language description of what the system does, step by step. Write this in the vocabulary of the problem, not of your file structure — it is a real match candidate for the requirements in Section 6.
5. Setup	Prerequisites with versions, install steps, environment variables, and whether paid API keys are required.
6. The primary path	Exactly what to run to see the thing work.
7. What you did not build, and why	The scope you deliberately cut, and what you would do with two more hours. Scope discipline is a scored criterion, so this is evidence, not an apology.
8. AI-use disclosure	Tools, models, and roughly what you used them for.
9. Attribution	Third-party components, and the provenance of any audio you used.

APPENDIX B

B Submission checklist

The first group before you submit, the second before you present.

	Item
	BEFORE YOU SUBMIT
☐	One primary concept, named in the README, with the friction ID and the outcome stated up front.
☐	Audio appears somewhere in the workflow — captured, generated, or transformed (MR-1).
☐	All six mandatory requirements (MR-1 to MR-6) have an obvious home in the repository.
☐	Each capability lives in one clearly named place rather than scattered across unrelated files.
☐	Names and comments use the vocabulary this document uses for the problem.
☐	README covers all nine sections in Appendix A.
☐	Section 3 of the README honestly separates what works from what is stubbed.
☐	Section 7 of the README says what you cut. You built one thing, not three.
☐	No secrets, tokens, or live credentials anywhere in the repository or its history.
☐	If your code needs credentials, an example environment file lists every variable.
☐	No real personal data, no non-consensual voice capture, no copyrighted commercial audio.
☐	Third-party components and any audio you used are attributed.
☐	The repository was created at or after kickoff, and commit timestamps show the work happened inside the window.
☐	The repository clones cleanly. If private, a correctly scoped read-only token is in the form.
☐	If you want a specific commit reviewed: it is named in the form. Otherwise we take main.
☐	Form submitted before the clock stopped — name, email, repo URL, branch if not main, token if private.
	BEFORE YOU PRESENT
☐	Your pre-baked result is saved and ready to show.
☐	You have said your two minutes out loud once, and it fit.
☐	You know what you will say is real and what is staged.
☐	You have the adapter your laptop needs for the room TV.
☐	You have plugged in and confirmed audio plays through the TV, not your laptop speakers.
☐	You are set up and ready to go when your slot comes up.

APPENDIX C

C Glossary

Term	Meaning
Banger	An original song produced by Business Bangerz for a specific business message. The unit of production and the unit of sale.
Jam Sesh	The free 20-minute intake consultation that opens a client relationship. The front door of the sales funnel and the source of most briefing material.
Jukebox	The members-only library of released bangers on the site, tagged by use case and genre.
Banger of the Week	The free weekly newsletter. Doubles as the sign-up mechanism for Jukebox access.
The Banger Loop	Listen → Create → Release → Listen Again. The target operating model described in the attached strategy report.
Brief	The structured record of what a song has to accomplish: objective, audience, required messages, stories, tone, references, pronunciations, and constraints.
Voice agent	For the purposes of this RFP, any system that conducts a spoken interaction — listening, reasoning, and responding — as part of a workflow. Whether it is realtime speech-to-speech or an assembled pipeline is your call.
VibeCheck	The automated review that matches your repository against the requirements in Section 6 before a human reads it. Specification: https://vibecraft.works/specs/vibecheck.md. See Section 10.2.